Peak Powerhouse Database Management System

Team: Peak Data (Group 3)

Arthur Adjamoglian and Min Deng

Step 3 Feedback from peer reviewer

Feedback from Paulo Mancilla

- **Environment & Setup:** Are the README instructions sufficient to install dependencies and run the application? Were any steps unclear, missing, or difficult to follow?

README instructions are clear and are sufficient to install dependencies. **However, I had run into errors when using "source peak_powerhouse_ddl.sql" because of this I had to use the DDL to INSERT the employees, locations, members, and sales into the DB.** You have two big errors one at line 25 and 202 your lines don't have space and this caused lots of issues when initially testing your code.

You had no space after the second '-' which didn't let me import into MySQL workbench correctly

---

-- Table `peak_ph_db.locations`

-- ----------

Instead of:

-- ----------

-- Table `peak_ph_db.locations`

-- ----------

Application-Tier Design: Does the Python class clearly act as an intermediary between test script code and the database? Are SQL operations encapsulated within application methods rather than exposed directly?

- The python class does act as an intermediary between the test script code, and the DB and SQL operations are encapsulated within the application.

- **READ Operations:** Does the application implement a READ (SELECT) operation for **all schema entities**? Do the SELECT queries make sense given the schema (e.g., appropriate joins, filtering, or ordering)?
  - All tables do have READ operations However the READ operations are currently limited to the retrieving information from individual tables with just the use of SELECT * FROM [Table Name] the biggest example being sales_details.
- **Intersection Table Operations:** Does the application correctly implement **CREATE, UPDATE, and DELETE** operations for the **intersection table**? When modifying the intersection table, does the logic preserve related entities appropriately (e.g., avoids delete anomalies)?
  - The table operations do correctly implement CREATE, UPDATE and DELETE operations for the intersection table sales_details.

- **Test / Driver Program:** Does the provided test or driver application successfully demonstrate all required operations? Is the CLI output sufficient to verify correctness without reading the code line-by-line?

o It does seem as the CREATE, UPDATE and DELTE operations are working correctly however my results are differing from the CLI.

=== Sales Details ===

[(1, 1, 1, 1), (2, 1, 2, 1), (3, 1, 3, 1)]

=== Testing INSERT ===

New sales_details_id: 4

Sales details after INSERT:

[(1, 1, 1, 1), (2, 1, 2, 1), (3, 1, 3, 1), (4, 2, 3, 2)]

=== Testing UPDATE ===

Rows updated: 1

Sales details after UPDATE:

[(1, 1, 1, 1), (2, 1, 2, 1), (3, 1, 3, 1), (4, 2, 3, 5)]

=== Testing DELETE ===

Rows deleted: 1

Sales details after DELETE:

[(1, 1, 1, 1), (2, 1, 2, 1), (3, 1, 3, 1)]

CLI:

=== Sales Details ===

[(7, 1, 1, 2), (8, 1, 2, 1), (9, 2, 1, 2), (10, 3, 3, 2)]

=== Testing INSERT ===

New sales_details_id: 12

Sales details after INSERT:

[(7, 1, 1, 2), (8, 1, 2, 1), (9, 2, 1, 2), (10, 3, 3, 2), (12, 2, 3, 2)]

=== Testing UPDATE ===

Rows updated: 1

Sales details after UPDATE:

[(7, 1, 1, 2), (8, 1, 2, 1), (9, 2, 1, 2), (10, 3, 3, 2), (12, 2, 3, 5)]

=== Testing DELETE ===

Rows deleted: 1

Sales details after DELETE:

[(7, 1, 1, 2), (8, 1, 2, 1), (9, 2, 1, 2), (10, 3, 3, 2)]

- **Code Quality & Design Suggestions:** Are method names, parameters, and return values clear and appropriate? Do you have suggestions for improving clarity, robustness, or organization of the application-tier code?

    - In terms of code quality and design names, parameters and returns are clear and appropriate.
- **Overall Feedback:** What aspects of this project are particularly strong? What specific improvements would you recommend before the final submission?
    - The strong suit of the project is the organization the files are well organized and have good naming methods. The only issue is I had an issue with sourcing the DDL. After fixing the Erros and even importing directly into MySql workbench I still had issues with my test and the CLI returning the same answers for Sales Details, and all the test inserts.

All work is my own, and no AI was used for this review

Actions Based on the Feedback - step 3

After reviewing the feedback from Paulo Mancilla, we found several things we need to improve in our project.

1. Fixed DDL import errors caused by malformed comment separators (Paulo)

Paulo reported that two incorrectly formatted comment lines (lines 25 and 202) caused errors when importing the DDL file.

We corrected the comment formatting and checked the rest of the DDL file for similar issues. We also re-imported the DDL into a clean database to make sure all tables and sample data loaded successfully.

2. Re-generated the CLI output from a clean database (Paulo)

Paulo noticed that his test results differed from our submitted CLI_OUT.txt, especially for the sales_details table. The difference may have been caused by different database states during testing.

After updating the DDL, we recreated the database, reran the driver program, and generated a new CLI_OUT.txt to make sure the output matches the final sample data and demonstrates the INSERT, UPDATE, and DELETE operations correctly.

3. Improved READ Queries (Paulo)

Paulo suggested using more meaningful READ queries instead of only using basic SELECT statements. We added a LEFT JOIN query to show the total sales for each gym location. This makes the query more useful and demonstrates how data from different tables can be combined.

Upgrades to the Draft Version

1. Improved the Installation Process (Additional Improvement)

We added a quick installation option to make it easier for users to set up the project. During testing, we found some issues with installing Python modules and loading sample data. We fixed these issues, updated the README instructions, and tested the installation again to make sure the database and CRUD operations worked correctly.

Step 2 Feedback from the TAs and peer reviewers

Feedback from Evan Neuenschwander

1, Does the schema present a physical model that follows the database outline and the ER logical diagram exactly? Looks like it, but the schema is exactly the ERD, which is...maybe a problem.
2, Is there consistency in a) naming between overview, outline, ER and schema entity/attributes b) entities plural, attributes singular c) use of capitalization for naming? There's only one major problem here, which is that roles have a different name for their PK in the outline compared to either other source.
3, Is the schema easy to read (e.g. diagram is clear and readable with relationship lines not crossed)? Mostly, but make some boxes wider! Some stuff gets cut off, and that's an issue.
4, Are intersection tables properly formed (e.g. two FKs and facilitate a M:N relationship)? Yeah, the intersection tables are properly made!
5, Does the sample data suggest any non-normalized issues, e.g. partial dependencies or transitive dependencies? It doesn't seem so, no.
6, Is the SQL file syntactically correct? This can be easily verified by using PhPMyAdmin and your CS 340 database (do not forget to take backup of your own database before you do this!) From inspection, looks nice!
7, In the SQL, are the data types appropriate considering the description of the attribute in the database outline? Same as the syntax, looks good!
8, In the SQL, are the primary and foreign keys correctly defined when compared to the Schema? Are appropriate CASCADE operations declared? Yup, all the PK's and FK's are here, but maybe add some CASCADEs? There are a couple tables that might have problems if you just deleted something...
9, In the SQL, are relationship tables present when compared to the ERD/Schema? They are there, so good work on that!
10, In the SQL, is all example data shown in the PDF INSERTED? Looks like it, yeah!
11, Is the SQL well-structured and commented (e.g. hand authored) or not (e.g. exported from MySQL)? Looks great, actually! Might potentially be exported, but it is commented and structured!
12, As a reviewer, clearly describe to what extent your feedback to the team was original (e.g. "all my work") or non-original (e.g. used AI tools per Code Citation Tips (opens in new tab)). All my own work.

Feedback from Christopher Long

Does the schema present a physical model that follows the database outline and the ER logical diagram exactly?

The schema shows a relationship between roles and employees that is not described in the employees entity in the outline. Additionally, the roles entity contains a key attribute whose name does not match between the schema and outline. Other than these two items, the schema, outline, and ERD all matched. Interestingly, the ERD was essentially the schema. Though I can’t think of anything in the assignment description that disallows this, it is a bit strange to just have the same diagram included twice. It may be worth considering replacing the diagram in the ERD section with a simplified diagram more akin to an ERD.

Is there consistency in a) naming between overview, outline, ER and schema entity/attributes b) entities plural, attributes singular c) use of capitalization for naming?

All naming is consistent between overview, outline, ER, and schema. Entity names were not capitalized, though stylistically it's easier when they are not. Snake case was used for attributes, which admittedly is my preference as well.

Is the schema easy to read (e.g. diagram is clear and readable with relationship lines not crossed)?

Some attributes are slightly cut off on the schema due to the breadth of the schema. This occurs on the employees, sales, sales_details, memberships, members, and products tables. I think it would be worthwhile making some adjustments and widening the tables to show the entirety of each attribute on all tables.

The schema is otherwise easy to read with no crossed relationship lines and all text clearly legible.

Are intersection tables properly formed (e.g. two FKs and facilitate a M:N relationship)?

Does the sample data suggest any non-normalized issues, e.g. partial dependencies or transitive dependencies?

The intersection table is properly formed and contains two foreign keys, 1 from each relationship, properly defining the M:N relationship between sales and products. There do not appear to be any issues with dependencies or normalization.

Is the SQL file syntactically correct? This can be easily verified by using PhPMyAdmin and your CS 340 database (do not forget to take backup of your own database before you do this!)

The SQL file is syntactically correct. I was able to import it to my mariadb instance without issues.

In the SQL, are the data types appropriate considering the description of the attribute in the database outline?

All of the data types are appropriate given their descriptions in the database outline.

In the SQL, are the primary and foreign keys correctly defined when compared to the Schema? Are appropriate CASCADE operations declared?

Primary and foreign keys appear to be properly defined. There are no cascade operations declared, and I'm not convinced this wouldn't cause issues when data is deleted. For example, though it should not happen, what if a member were to be deleted from the members table?

In the SQL, are relationship tables present when compared to the ERD/Schema?

In the SQL, all relationship tables are present, and after importing the tables appear in the output of show tables.

In the SQL, is all example data shown in the PDF INSERTED?

All of the data shown is present in the sql. However, the example data consists of screenshots of the select query outputs. While efficient, this yielded some difficult to see screenshots due to the size of the text when made to fit in the PDF. It may be good to find an easier to read way of conveying the sample data.

Is the SQL well-structured and commented (e.g. hand authored) or not (e.g. exported from MySQL)?

The SQL is well structured and commented. Comments appear before each major section of code. The structure is what I'd expect from a dump, which is what the file appears to be based on its structure and the header at the top.

As a reviewer, clearly describe to what extent your feedback to the team was original (e.g. "all my work") or non-original (e.g. used AI tools per Code Citation Tips).

All feedback is original and no AI was used.

Feedback from Mina Go

Does the schema present a physical model that follows the database outline and the ER logical diagram exactly? Is there consistency in a) naming between overview, outline, ER and schema entity/attributes b) entities plural, attributes singular c) use of capitalization for naming?

It looks like the schema is a copy of the ERD. I'm guessing you gave your ERD more detail than you needed because it makes it easier to add detail to your database outline then you realized you accidentally created a schema- that's what happened to me. The names between your outline, ERD, and schema are consistent, but the capitalization is not. For example, your outline says 'Sales' but your ERD/schema says 'sales'. You do consistently use snake case for your attributes. I also noticed a lot of your attributes contain the entity name. This does not break any rule, but realize that if you have to reference an attribute you will get some very long names that will be annoying to type and tedious to look at such as 'memberships.membership_name'. I would strongly recommend making your attribute names less redundant.

Is the schema easy to read (e.g. diagram is clear and readable with relationship lines not crossed)?

The diagram is clear and readable, and the relationship lines do not cross. I would appreciate it if you spread out the tables more so the relationship lines don't have as many bends. It takes me an extra second to identify your M:N relationships/intersection tables because those bends pull my eye in a bunch of different directions.

Are intersection tables properly formed (e.g. two FKs and facilitate a M:N relationship)?

The intersection tables are properly formed as in they have two FKs, but I think you're trying to do too much at once and overlapping your intersection tables with your entities. I don't know that this actually breaks any rules, but it definitely takes more time to understand your schema because of that.

Does the sample data suggest any non-normalized issues, e.g. partial dependencies or transitive dependencies?

The sample data does not suggest any non-normalized issues from what I can tell.

Is the SQL file syntactically correct? This can be easily verified by using PhPMyAdmin and your CS 340 database (do not forget to take backup of your own database before you do this!)

The SQL file does not appear to have any major errors.

In the SQL, are the data types appropriate considering the description of the attribute in the database outline?

You don't actually describe attributes in your database, but your attribute names are relatively self-explanatory. The data types seem appropriate given their limited descriptions.

In the SQL, are the primary and foreign keys correctly defined when compared to the Schema? Are appropriate CASCADE operations declared?

Primary and foreign keys are correctly defined. Appropriate CASCADE operations are declared.

In the SQL, are relationship tables present when compared to the ERD/Schema?

In the SQL, is all example data shown in the PDF INSERTED?

Almost all of the example data shown in the PDF is inserted except the 1:M and M:N relationship examples.

Is the SQL well-structured and commented (e.g. hand authored) or not (e.g. exported from MySQL)?

The SQL is well-commented. Comments separate each step of creating the database and inserting sample data.

As a reviewer, clearly describe to what extent your feedback to the team was original (e.g. "all my work") or non-original (e.g. used AI tools per Code Citation Tips).

All my work.

Feedback from Razeel Hassan Ibrahim Rasheed（TA）

Good work on the draft! Just a few things to fix before the final submission.

Please double-check the many-to-many relationship between Sales and Products, as a constraint in your SQL currently prevents the same product from appearing in different sales. Also, update the sample data to demonstrate this relationship properly and make sure the sale totals match the items purchased.

Please make sure the database outline, schema, and SQL all match, particularly after adding the Roles table. The 3NF explanation could also be a little clearer.

Some of the diagrams and sample-data screenshots are quite small, so please enlarge them to make them easier to read.

Overall, nice progress!

Actions Based on the Feedback - step 2

After reviewing the feedback from our TA, Razeel Hassan Ibrahim Rasheed, and three classmates (Evan Neuenschwander, Christopher Long, and Mina Go), we found several things we need to improve in our project.

1. Fixed the Many-to-Many Relationship (TA)

We removed the UNIQUE constraint on products_product_id because it stopped the same product from being used in different sales. We also changed the primary key to sales_details_id and added a new UNIQUE constraint using sales_transaction_id and products_product_id. Now

the same product can be sold in different transactions, but it cannot be listed twice in the same sale.

2. Updated Sample Data and Sales Totals (TA, Christopher, Mina)
We changed our sample data to show the many-to-many relationship between Sales and Products. We added examples where one sale has more than one product, and the same product can be sold in different sales. We also fixed the sale totals to make sure the amounts match the products purchased, including sales tax.

3. Improved Database Consistency (TA, Evan, Christopher)
We updated our database outline to match our SQL after adding the Roles table. We changed employee_role to roles_role_id in the Employees table and fixed the primary key name in the Roles table. We also changed the phone number data types to VARCHAR(20) and updated some attributes and relationships to make sure they match our SQL.

4. Improved the Normalization Explanation (TA)
We updated our explanation of 1NF, 2NF, and 3NF to make it easier to understand. We also fixed our explanation about addresses because keeping addresses in the same table does not automatically mean the database is not in 3NF.

5. Improved the ERD, Schema Diagram, and Sample Data (TA, Evan, Christopher, Mina)
We made a new ERD with a simpler layout to make the relationships easier to see. We also updated our Schema Diagram to match the SQL and made the tables easier to read. We exported a clearer image instead of using screenshots. We also updated our sample data screenshots to make the information easier to read.

Step 1 Feedback by the TAs and peer reviewer

Feedback from Pavel Gorshenkov

This database back-end implementation is perfect introduction into transactional DBMS concepts of entities, attributes, constraints and relationships between these characteristics. Designing a system for a gym business is also a great way to conceptualize these tools and to put them into use. This business has a clear need for managing a database with three locations and a unique set of employees, equipment, members, and transactions. You have clearly outlined the gym-related services and a general overview of information storing and management.

In terms of raw numbers, you specified a gym-member count, number of gym locations and monthly transactions. Transactions that include monthly membership plans, training sessions, and other gym-related items. Good design choices were made in centralizing the gym location entity which then shares a location foreign key id with employees, sales, members and equipment. This is effectively visualized in the ERD.

Entities Listed: Locations, Equipment, Roles, Employees, Memberships, Members, Items, Sales, sales_has_items, Sessions.

It is worth mentioning, that sales_has_items should be labeled as an associative/relationship table as it is meant to facilitate a many-many relationship between items and sales. Roles:

This is a wise design choice as a role change or update would result in a change in one row instead of sorting through every employee table updating each role string. The Employees table could be populated with more attributes consistent with a modern workplace title. In most positions of employment it is common to include a residence address, email address, phone number, date of employment, etc.

In the ERD and in the database design it would make more sense to have a direct, one-to-many relationship between 'sales_has_items' and 'sessions'. Does bridging the gap between 'sales_has_items' and 'sessions' feel less structurally awkward from a structural business point of view?

The outline correctly formats and presents a well defined 1:M relationship under each entity. At least one M:M relationship is also present between sales and items. It might be worth mentioning a M:M relationship between members and sessions that may exists that isn't defined.

It's important to note that the sales entity has a transaction id and member id, quantity, etc. There is no way to differentiate between a session and other gym items. A null-able session id should be included in the sales table and a quantity attribute should only exist in sales_has_items to make a distinction between which sale of what gym product is made. Naming conventions are sensible and consistent with plural entities and singular attributes. All feedback is student-generated and human-authored.

Feedback from Ezri Martinez

Does the overview describe what problem is to be solved by a website with DB back end? If yes, summarize. If not, what changes would better support describing the problem to be solved? Yes, the problem is stated as information management of various aspects of the business. Although, I think it would be helpful to be more specific on something that requires information management rather than generalizing information management as a whole. This could limit the potential for over engineering.

Does the overview list specific facts? If yes, summarize what the facts illustrate about the proposed DB solution. If not, what facts would better support illustrating the scope and scale of the proposed DB solution?

Yes, it lists that approximately 1000 sale transactions are made per month, there are approximately 2000 members, and there are three gym locations. This defines the need for an information management system by providing the scale of business operations. These facts also signify the need for a central information management service due to multiple locations. Are at least four entities described, and does each one represent a single idea to be stored as a list? If yes, summarize. If not, based on the course material, what changes can you suggest to improve?

Yes, there are 10 entities and all represent a single idea. The relations are: Locations tracks information of each gym locations, Equipment tracks information about equipment availability, Roles tracks employment roles, Employees tracks employee statistics, Memberships track membership plan information, Members tracks member statistics, Items tracks items/services availability, Sales tracks sale transactions, sale_has_items connects Sales and Items, and Sessions tracks items linked to scheduled sessions.

Does the outline of entity details describe the purpose of each, list attribute datatypes and constraints, and describe relationships between entities? If yes, summarize. If not, based on the course material, what changes can you suggest to improve?

Yes, the entity details have a purpose, datatypes, constraints and describe the relationships between entities. The list of entities has fields for each of these as a

description, datatypes, and relationships. However, I noticed some datatypes might be better suited as another datatype. For example, member_phone could be varchar rather than int. Also I think that any currency cost attributes should be decimal rather than int.

Are 1:M relationships correctly formulated? Is there at least one M:M relationship? Does the ERD present a logical view of the database? If yes, summarize. If not, based on the course material, what changes can you suggest to improve?

Yes, the 1:M relationships are correct and there is one M:M relationship. The 1:M relationships are as follows: locations to equipment, employees, members, and sales, roles to employees, and items to sessions. The M:M relation is between sales and items through sales_has_items with a PK. The ERD makes sense and matches the outline. Is there consistency in a) naming between overview and entity/attributes b) entities plural, attributes singular c) use of capitalization for naming? If yes, summarize. If not, based on the course material, what changes can you suggest to improve?

a) Yes, the namings in the overview align with the entities.
b) Yes, entities are plural and attributes are singular.

c) No, sales_has_items is lowercase while the rest of the entities are capitalized. To fix this, I'd suggest making all entities lowercase to match the capitalization of the ERD.

Originality: The response to the questions above are largely my own. Claude was used to review clarity, flow, and grammar.

Anthropic. (October 12, 2026). Claude (Opus 5.5) [Large language model]. https://claude.ai. Scope: Used to review my draft answers for clarity, grammar, and writing flow. Prompts: I asked Claude to review my responses to questions, correct errors, and clarify missing elements.

Feedback from Jacob Gervais

Does the overview describe what problem is to be solved by a website with DB back end? If yes, summarize. If not, what changes would better support describing the problem to be solved?

Your introductory paragraph does explicitly describe what problem a database will be solving: "The system needs to manage information about members, employees, equipment, membership plans, training sessions, and sales. A database-backed website will help the business organize this information and keep track of the relationships between members, employees, locations, memberships, and sales."

I think one or two sentences describing the current hardship or main difficulties prior to having a DB backend would make this introductory paragraph stronger.

Additionally, while one paragraph could be sufficient, splitting your paragraph into 2 smaller (and more focused) paragraphs could be a good idea to improve readability. One paragraph describing the business and the scale of transactions per year, members, etc., and another paragraph describing the main problem(s).

Does the overview list specific facts? If yes, summarize what the facts illustrate about the proposed DB solution. If not, what facts would better support illustrating the scope and scale of the proposed DB solution?

The overview does list a few specific facts. I see that there are 2000 members and 1000 transactions per month. This information is good but I cant help but feel there should be more. For instance, there should be a mention of membership cancellations and renewals

maybe. In total, these facts illustrate for me a small-to-medium scale database would be necessary to handle the day-to-day transactions at this gym. With 3 locations, the database will need to be hosted online in order to synchronize data between the 3 locations.

Are at least four entities described, and does each one represent a single idea to be stored as a list? If yes, summarize. If not, based on the course material, what changes can you suggest to improve?

There are 10 entities described and each one represents a single listable object / event. There is Locations, Equipment, Roles, Employees, Memberships, Members, Items, Sales, sales_has_items, and Sessions.

These are all represent a single idea stored as a list and are logically named for the most part. I am just a little confused by a couple things.
Why is “sales_has_items” all lowercase?

What is “Sessions”? Even after reading the description, I really am not sure what it is supposed to be representing.

For readability, I would consider adding more detail to your description of each entity if they are not super straight-forward.

Does the outline of entity details describe the purpose of each, list attribute datatypes and constraints, and describe relationships between entities? If yes, summarize. If not, based on the course material, what changes can you suggest to improve? The purpose of each entity is described, although fairly briefly. I think one additional sentence would be helpful for a few of them.

Each entity does list attributes, datatypes, and constraints. The only thing I would argue with here is that anything money related probably shouldn’t be an INT (e.g. membership_monthly_charge should probably be decimal (2,2))

The relationships between entities are very well described. This helps illustrate the flow of the database. No additional writing is required here.

Are 1:M relationships correctly formulated? Is there at least one M:M relationship? Does the ERD present a logical view of the database? If yes, summarize. If not, based on the course material, what changes can you suggest to improve?

The relationships described and on the diagram appear to be correctly formulated. There is a M:N relationship between sales and items. This relationship is facilitated by the sales_has_items intersection table.

The ERD looks great with all relationships looking proper as described above. All attributes are visible and there are symbols representing each attribute type which helps with understanding the ERD.

Is there consistency in a) naming between overview and entity/attributes b) entities plural, attributes singular c) use of capitalization for naming? If yes, summarize. If not, based on the course material, what changes can you suggest to improve? The naming between the overview and the ERD is consistent. I believe it was generated automatically using MySQL workbench.

The entities are all plural, while the attributes are all singular.

The entities are all lowercase in the ERD. They should begin with a capital. The attributes use snake casing and are all singular.

Overall, the ERD looks great and should not take too long to fix the capitalization issue. **As a reviewer, clearly describe to what extent your feedback to the team was original (e.g. "all my work") or non-original (e.g. used AI tools per Code Citation Tips).** This review was done using zero AI or outside resources whatsoever.

Feedback from Thien Ngan Nguyen

The overview clearly explains why Peak Powerhouse would benefit from a database backed website. The business has three locations and needs to keep information about members, employees, equipment, memberships, sessions and sales together in one system. The facts about having around 2000 members and 1000 sales transactions each month also help show the size of the database and why keeping the information organized is important. There are more than four entities and the main entities each represent a clear type of information that needs to be stored, such as Locations, Employees, Members, Items and Sales. The outline also explains the purpose of the entities and includes attributes, datatypes, constraints and relationships. One thing I noticed is that manager_id is included in Locations but it is not marked as a foreign key or explained as a relationship so I think that connection should be clarified. Some money related attributes such as price and membership charges may also work better as DECIMAL instead of INT. The 1:M relationships generally make sense and there is an M:N relationship between Sales and Items that is resolved using sales_has_items. I would double check the key for sales_has_items, though because only sales_transaction_id appears to be marked as the primary key. Since one sale can contain several different items, I think both foreign keys could form a composite primary key. I would also move quantity from Sales into sales_has_items since each item in the same sale could have a different quantity. The ERD otherwise matches the written outline and gives a logical view of the database. Naming is mostly consistent with plural entity names and singular attributes, although sales_has_items uses a different capitalization style from the other entity names. Overall, I think the design is easy to follow and the main improvements I would suggest are clarifying the manager relationship and checking the structure of the Sales to Items intersection table. Originality: I reviewed the project draft myself and used Grammarly to check my spelling

Feedback from Razeel Hassan Ibrahim Rasheed(TA)

Good work on your proposal! Your overview is clear, and you've included enough numerical examples. Your database outline and ERD look good overall. One recommendation is to simplify the scope of your project, since you currently have 10 tables, which could be difficult to manage within the term. Around 5–7 tables should be enough for this project, so consider focusing on the most important features. Also, since a sale can contain multiple items, consider moving quantity to sales_has_items. Finally, clarify whether manager_id in Locations refers to an employee, since that relationship isn't shown in the ERD. Just a reminder that the overall project requires a Python class as the application interface to your database, rather than a website.

Actions Based on the Feedback - step 1

After reviewing the feedback from our four classmates and the TA, we discussed their suggestions and made several changes to improve our database design.

1. Updated Data Types (Ezri, Jacob, Thien)

We changed money-related attributes from INT to DECIMAL, including product prices, product costs, and membership monthly charges. We also changed phone number attributes from INT to VARCHAR(10) because our gym operates in the United States and we plan to store phone numbers as 10-digit numbers without formatting.

2. Improved the Sales Structure (Pavel, Thien, TA)

We renamed sales_has_items to sales_details and moved quantity from Sales to Sales Details. This allows each product in a transaction to have its own quantity. We also added sales_details_id as the primary key for the Sales Details table.

3. Simplified the Database Structure (TA)

Based on the TA's suggestion, we reduced our database from 10 tables to 7 tables to make the project more manageable. We removed Equipment, Roles, and Personal_training_sessions to focus on the main features of our gym business.

Instead of having a separate Roles table, we added employee_role to Employees to store employee job titles. We also decided to include personal training services in Products, so members can still purchase training sessions without having a separate table for scheduling appointments.

4. Added More Employee Information (Pavel)

Based on Pavel's suggestion, we added employee_phone, employee_email, and employee_address to the Employees table to store more useful employee information.

5. Simplified the Location Manager Relationship (Thien, TA)

Thien and the TA suggested clarifying the manager_id attribute in Locations. After reviewing our database design, we decided to remove this attribute to keep the structure simpler. Since the Employees table already includes employee_role and locations_location_id, we can still identify employees' job roles and their assigned gym locations without creating an additional manager relationship.

6. Improved the Project Overview (Ezri, Jacob, TA)

We rewrote the Project Overview to explain our gym's business model and operations more clearly. We added details about monthly memberships, personal training services, retail products, employees, and sales transactions. We also described the challenges of managing information across three locations and how a centralized database could help solve these problems. Following the TA's instructions, we removed the reference to a database-backed website.

Suggestions We Decided Not to Follow

1. Entity Naming Consistency (Ezri, Jacob, Thien)

Some reviewers suggested changing the capitalization of our entity names. After discussing this, we decided to keep all entity names lowercase in our ERD. We think lowercase names are easier to read and work with, especially when writing SQL queries. We also updated our entity names to follow a consistent lowercase naming convention, so this still addresses the main concern about naming consistency.

2. Adding a Direct Relationship Between Sales Details and Personal Training Sessions (Pavel)

Pavel suggested adding a direct 1:M relationship between sales_has_items (now sales_details) and Sessions.
We decided not to add this relationship because we removed the Sessions table after simplifying our database based on the TA's feedback. Personal training is now treated as a service in Products, and Sales Details records its purchase. Our database will focus on sales transactions rather than appointment scheduling.

3. Adding an M:N Relationship Between Members and Sessions (Pavel)

Pavel also suggested considering an M:N relationship between Members and Sessions.

We decided not to add this relationship because our project no longer includes a separate Sessions table. Our gym focuses on one-on-one personal training rather than group classes, and we only need to record the purchase of these services. Adding a junction table to manage session bookings would make our project more complicated than necessary.

4. Including Membership Cancellations and Renewals in the Overview (Jacob)

Jacob suggested mentioning membership cancellations and renewals to provide more specific business information in our Overview.

We decided not to include these details because our current database focuses on managing active members and their membership plans rather than membership cancellation and renewal history. We wanted to keep the Overview focused on the features included in our simplified database design.

Upgrades to the first Draft Version

In addition to the changes based on peer and TA feedback, we made several improvements to our original database design.

1. Added Member Join Date

We added member_join_date (DATETIME) to the Members table so the gym can track when each member joined.

2. Added Employee First and Last Names

We added employee_fname (VARCHAR(50)) and employee_lname (VARCHAR(50)) to the Employees table to store employee names and make it easier to identify individual employees.

3. Added Sales Total

We added sale_total (DECIMAL(8,2)) to the Sales table to record the total amount of each transaction.

4. Improved the ERD Layout

After reducing our database from 10 tables to 8 tables, we rearranged the remaining tables in our ERD to make the relationships easier to see and understand. We placed related tables

closer together and reduced overlapping relationship lines to improve readability.

Upgrades to the second Version

1. Added roles table

We added a roles table to reduce data redundancy of multiple employees having the same role name in the employees table. The downside is we have an additional table that we now have to manage.

2. Increased length of phone field

The initial use of VARCHAR(11) could potentially lead to issues based on how phone numbers may be formatted. Thus based on Razeel's (TA) recommendation we lengthen this to VARCHAR(20) for employee_phone in the employees table, and member_phone in the members table.

Overview

Peak Powerhouse is a West Coast regional gym business operating three locations, offering 24/7 gym access to its members. Across all three locations, the business employs approximately 60–70 staff members and serves around 2,000 active members.

Peak Powerhouse operates on a monthly subscription model with different membership plans that provides 24/7 gym access. In addition to their memberships, members can purchase add-on services, such as 50-minute personal training sessions, and retail products, including gym merchandise, beverages, and fitness snacks. Both personal training services and retail products are recorded in the Products entity.

Peak Powerhouse handles membership information and daily purchases of products and services across its three locations. Managing information across multiple locations can be challenging without a centralized database. For example, member records may become inconsistent between locations, and employees may have difficulty tracking membership information and sales transactions.

To address these challenges, Peak Powerhouse will use a centralized database to organize and manage its business information. The system will help maintain member and employee records, manage membership plan information, track sales transactions and purchased products or services, and keep information consistent across all three locations.

Database Outline

The following entities describe the main parts of the Peak Powerhouse database, including their attributes, data types, constraints, and relationships.

Locations

The Locations entity stores information about each Peak Powerhouse gym location, including its address.

- location_id: INT, auto_increment, NOT NULL, PK
- location_address: VARCHAR(255), NOT NULL

Relationships:

- There is a 1:M relationship between Locations and Employees. One Location can have many Employees, and locations_location_id is used as a FK in Employees. ● There is a 1:M relationship between Locations and Members. One Location can have many Members, and locations_location_id is used as a FK in Members.
- There is a 1:M relationship between Locations and Sales. One Location can have many Sales transactions, and locations_location_id is used as a FK in Sales.

Employees

The Employees entity stores information about Peak Powerhouse gym employees, including their names, contact information, job roles, and assigned locations.

- employee_id: INT, AUTO_INCREMENT, NOT NULL, PK
- employee_phone: VARCHAR(20)
- employee_email: VARCHAR(100)
- employee_address: VARCHAR(255)
- locations_location_id: INT, NOT NULL, FK
- roles_role_id: INT, NOT NULL, FK
- employee_fname: VARCHAR(50)
- employee_lname: VARCHAR(50)

Relationships:

- There is a 1:M relationship between Locations and Employees. One Location can have many Employees, and locations_location_id is used as a FK in Employees.
- There is a 1:M relationship between Roles and Employees. One Role can apply to many Employees, and roles_role_id is used as a FK in Employees.
- There is a 1:M relationship between Employees and Sales. One Employee can be associated with many Sales transactions, and employees_employee_id is used as a FK in Sales.

Memberships

The Memberships entity stores information about the monthly membership plan offered by Peak Powerhouse, including its name, description, and monthly charge.

- membership_id: INT, auto_increment, NOT NULL, PK
- membership_name: VARCHAR(45)
- membership_description: VARCHAR(255)
- membership_monthly_charge: DECIMAL(8,2)

Relationships:

- There is a 1:M relationship between Memberships and Members. One Membership Plan can have many Members, and memberships_membership_id is used as a FK in Members.

Members

The Members entity stores information about Peak Powerhouse gym members, including their personal information, contact details, membership plans, and join dates.

- member_id: INT, NOT NULL, PK
- member_fname: VARCHAR(50)
- member_lname: VARCHAR(100)
- member_address: VARCHAR(255)
- member_dob: DATE
- member_phone: VARCHAR(20)
- member_email: VARCHAR(255)
- locations_location_id: INT, NOT NULL, FK
- memberships_membership_id: INT, NOT NULL, FK
- member_join_date: DATETIME

Relationships:

- There is a 1:M relationship between Locations and Members. One Location can have many Members, and locations_location_id is used as a FK in Members.
- There is a 1:M relationship between Memberships and Members. One Membership Plan can have many Members, and memberships_membership_id is used as a FK in Members.
- There is a 1:M relationship between Members and Sales. One Member can have many Sales transactions, and members_member_id is used as a FK in Sales.

Products

The Products entity stores information about products and services available for purchase at Peak Powerhouse, including gym merchandise, beverages, fitness snacks, and personal training services.

- product_id: INT, NOT NULL, PK, AUTO_INCREMENT
- product_name: VARCHAR(100)
- product_description: VARCHAR(255)
- product_price: DECIMAL(8,2)
- product_cost: DECIMAL(8,2)

Relationships:

- There is a M:N relationship between Sales and Products, managed through the Sales_details entity. One Sale can include multiple Products, and one Product can appear in multiple Sales transactions.
- The products_product_id attribute is used as a FK in Sales_details, referencing Products.product_id.

Sales

The Sales entity stores information about sales transactions at Peak Powerhouse, including the total amount of each sale, the member making the purchase, the employee handling the transaction, and the gym location where the sale takes place.

- transaction_id: INT, NOT NULL, PK, AUTO_INCREMENT
- sale_total: DECIMAL(8,2)
- members_member_id: INT, NOT NULL, FK
- employees_employee_id: INT, NOT NULL, FK
- locations_location_id: INT, NOT NULL, FK

Relationships:

- There is a 1:M relationship between Members and Sales. One Member can have many Sales transactions, and members_member_id is used as a FK in Sales. ● There is a 1:M relationship between Employees and Sales. One Employee can handle many Sales transactions, and employees_employee_id is used as a FK in Sales. ● There is a 1:M relationship between Locations and Sales. One Location can have many Sales transactions, and locations_location_id is used as a FK in Sales.
- There is a 1:M relationship between Sales and Sales_details. One Sale can contain multiple Sales_details records, and sales_transaction_id is used as a FK in Sales_details.

Sales_details

The Sales_details entity stores information about the individual products or services included in each sales transaction, including the quantity purchased.

- sales_details_id: INT, NOT NULL, PK, AUTO_INCREMENT
- sales_transaction_id: INT, NOT NULL, FK
- products_product_id: INT, NOT NULL, FK
- quantity: INT

Relationships:

- There is a 1:M relationship between Sales and Sales_details. One Sale can have multiple Sales_details records, and sales_transaction_id is used as a FK in Sales_details.
- There is a 1:M relationship between Products and Sales_details. One Product can appear in multiple Sales_details records, and products_product_id is used as a FK in Sales_details.
- Sales_details serves as the intersection table for the M:N relationship between Sales and Products.

Roles

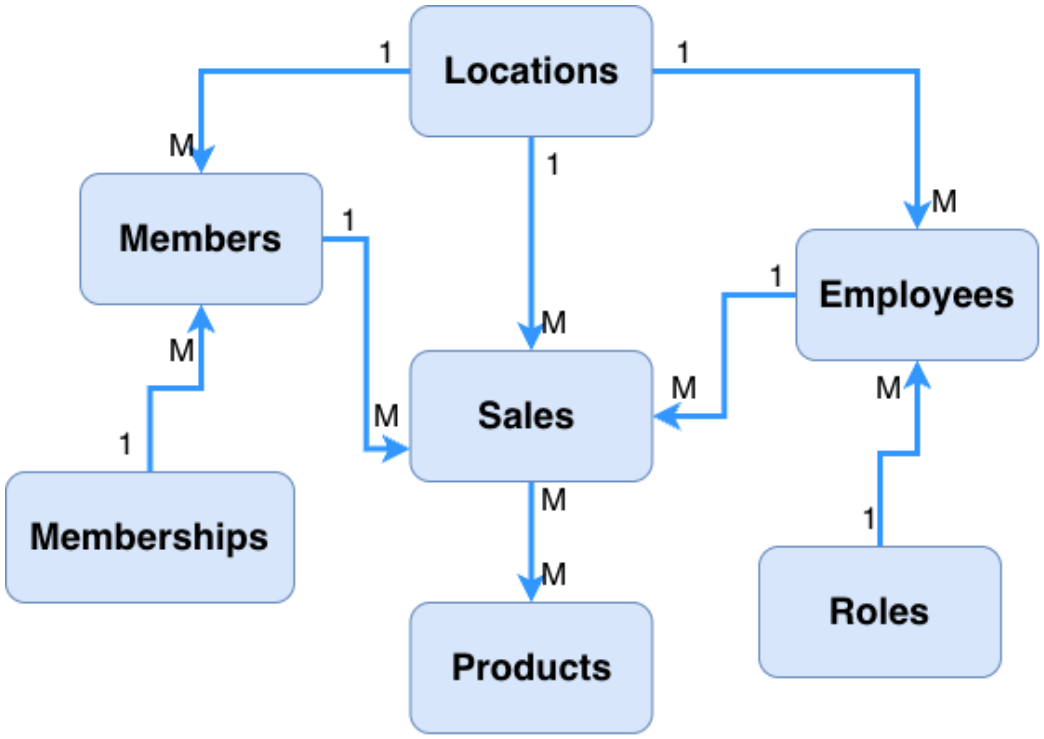
The roles entity stores information about the different roles/job titles an employee will have. The purpose of this table is to reduce the data redundancy of repeating similar roles in the employee table. Each employee will have at most one role.

- role_id: INT, AUTO_INCREMENT, NOT NULL, PK
- role_name: VARCHAR(45)

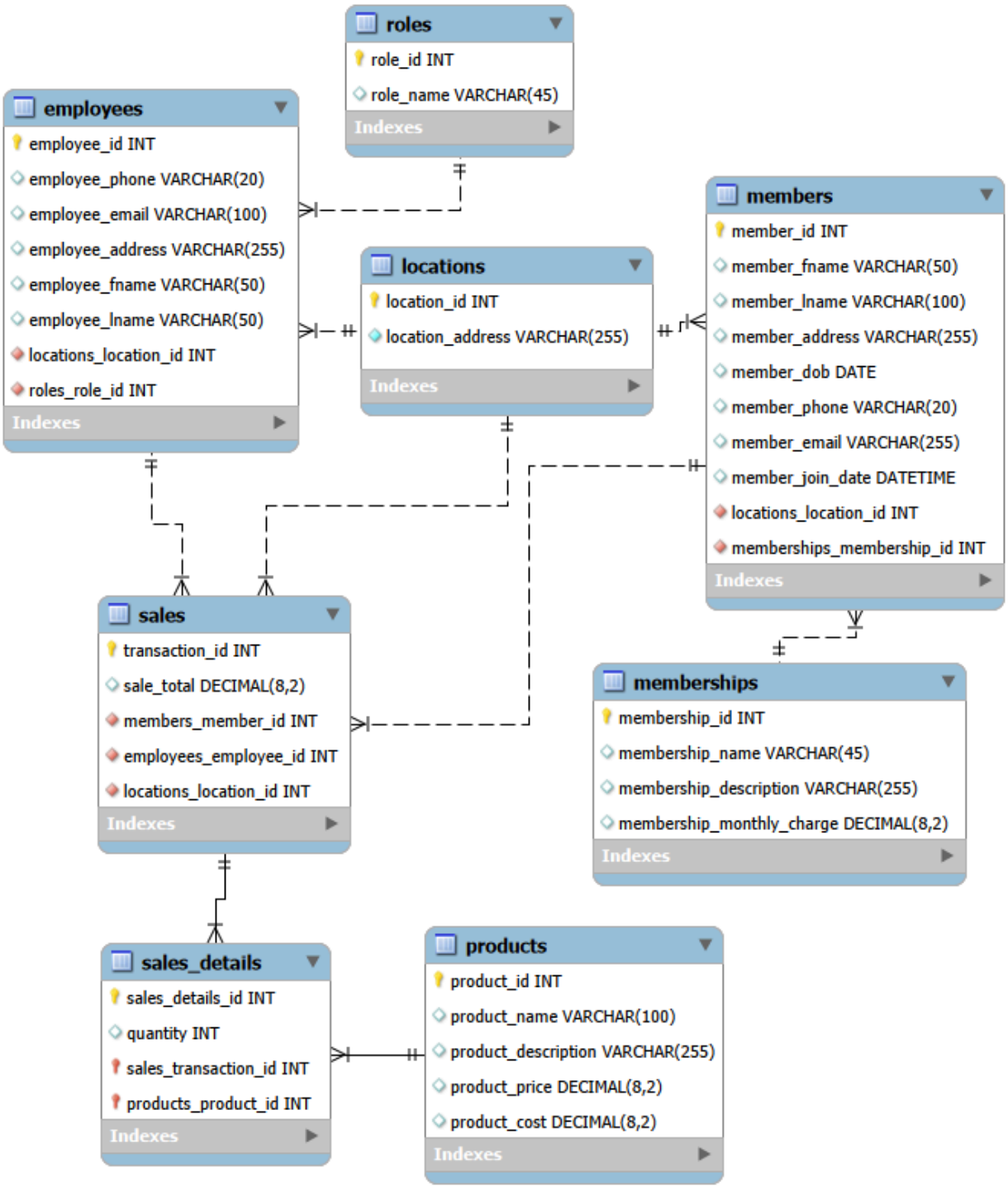
Relationships:

- There is a 1:M relationship between Roles and Employees. One Role can apply to multiple different employees. roles_role_id is a foreign key in the employees table.

Entity-Relationship Diagram



Schema Diagram



First Normal Form (1NF)

Our tables follow 1NF because each table has a primary key, and each column stores one value. There are no repeating groups in our tables.

Second Normal Form (2NF)

Our tables follow 2NF because they are already in 1NF, and all non-key attributes depend on the primary key. We also use sales_details_id as the primary key in the Sales Details table, so there are no partial dependencies.

Third Normal Form (3NF)

Our tables follow 3NF because they are already in 2NF, and there are no transitive dependencies between non-key attributes. We also added the Roles table so that we do not have to store the same role names for multiple employees. The Employees table uses roles_role_id to reference the Roles table instead of storing the role names directly.

Sample Data

```
mysql> SELECT * FROM locations;
+-------------+-----------------------------------------------------+
| location_id | location_address                                    |
+-------------+-----------------------------------------------------+
|           1 | 1060 W Alameda Ave, Burbank, CA 91506               |
|           2 | 6150 Laurel Canyon Blvd, North Hollywood, CA 91506 |
|           3 | 12811 Victory Blvd, Valley Glen, CA 91506           |
+-------------+-----------------------------------------------------+
```

```
mysql> SELECT * FROM roles;
+---------+------------------------+
| role_id | role_name              |
+---------+------------------------+
|       1 | Front Desk Associate   |
|       2 | General Manager        |
|       3 | Assistant Manager      |
|       4 | Maintenance Technician |
|       5 | Janitor                |
+---------+------------------------+
```

```
mysql> SELECT * FROM employees;
+-------------+----------------+-------------------------+-------------------------+---------------+---------------+----------------------+---------------+
| employee_id | employee_phone | employee_email          | employee_address        | employee_fname | employee_lname | locations_location_id |
roles_role_id |
+-------------+----------------+-------------------------+-------------------------+---------------+---------------+----------------------+---------------+
|           1 | 4599380912     | nruggiero0@cbc.ca       | 08 Kings Alley          | Nixie         | Ruggiero      |                    1 |             1 |
|           2 | 6225078723     | eempleton1@behance.net  | 8257 Weeping Birch Place | Ebony        | Empleton      |                    1 |             2 |
|           3 | 7687192767     | awhitcher2@guardian.co.uk | 1017 Debs Court       | Alie          | Whitcher      |                    1 |             3 |
|           4 | 3335204235     | mmadill3@booking.com    | 836 Declaration Trail   | Myrtia        | Madill        |                    1 |             4 |
|           5 | 2795868841     | jbohlens4@etsy.com      | 5 Petterle Alley        | Jolene        | Bohlens       |                    1 |             5 |
+-------------+----------------+-------------------------+-------------------------+---------------+---------------+----------------------+---------------+
```

```
mysql> SELECT * FROM memberships;
+---------------+-----------------------+-----------------------------------------------------------------+--------------------------+
| membership_id | membership_name       | membership_description                                          | membership_monthly_charge |
+---------------+-----------------------+-----------------------------------------------------------------+--------------------------+
|             1 | Single-Club Membership | Access to only one Powerhouse Gym location.                    |                    49.99 |
|             2 | Multi-Club Membership  | Access to all Powerhouse Gym locations.                        |                    69.99 |
|             3 | Trial Membership       | Trial membership for first timers. Free access for one week.   |                     0.00 |
+---------------+-----------------------+-----------------------------------------------------------------+--------------------------+
```

```
mysql> SELECT * FROM members;
+-----------+-------------+-------------+------------------------+------------+-------------+-----------------------------+---------------------+----------------------+-----------------
---------+
| member_id | member_fname | member_lname | member_address         | member_dob | member_phone | member_email                | member_join_date    | locations_location_id | memberships_membership_id |
+-----------+-------------+-------------+------------------------+------------+-------------+-----------------------------+---------------------+----------------------+-----------------
---------+
|         1 | Amalia      | Laddle      | 6 Nobel Lane           | 1951-02-15 | 5209532001  | aladdle0@twitter.com        | 2026-10-07 00:00:00 |
1 |                     1 |
|         2 | Codee       | Oscroft     | 88212 Stone Corner Road | 1996-08-22 | 2171827678  | coscroft1@mapy.cz           | 2026-03-29 00:00:00 |
1 |                     1 |
|         3 | Alain       | Spurr       | 8 Quincy Terrace       | 1957-04-02 | 2383921207  | aspurr2@deliciousdays.com   | 2025-10-01 00:00:00 |
2 |                     2 |
|         4 | Katti       | Antonignetti | 92838 Sugar Terrace    | 1970-06-26 | 2141214028  | kantonignetti3@slideshare.net | 2026-08-19 00:00:00 |
2 |                     2 |
|         5 | Bambie      | Higgoe      | 2 Monument Junction    | 1987-10-14 | 9476409290  | bhiggoe4@nyu.edu            | 2026-03-05 00:00:00 |
3 |                     3 |
+-----------+-------------+-------------+------------------------+------------+-------------+-----------------------------+---------------------+----------------------+-----------------
---------+
```

```
mysql> SELECT * FROM products;
+------------+-----------------+-----------------------------------------------+---------------+--------------+
| product_id | product_name    | product_description                           | product_price | product_cost |
+------------+-----------------+-----------------------------------------------+---------------+--------------+
|          1 | Energy Drink    | Super caffeinated sugary power bomb of a drink! |          4.99 |         1.75 |
|          2 | Pre-Workout Shot | Dry scooped directly into the customer's mouth. |          2.99 |         1.00 |
|          3 | Snickers Bar    | You are not you when you are hungry.          |          2.99 |         0.99 |
+------------+-----------------+-----------------------------------------------+---------------+--------------+
```

```
mysql> SELECT * FROM sales;
+----------------+------------+-------------------+----------------------+----------------------+
| transaction_id | sale_total | members_member_id | employees_employee_id |
locations_location_id |
+----------------+------------+-------------------+----------------------+----------------------+
|              1 |      14.30 |                 1 |                    1 |                    1 |
|              2 |      11.00 |                 2 |                    1 |                    1 |
|              3 |       6.59 |                 3 |                    1 |                    1 |
+----------------+------------+-------------------+----------------------+----------------------+
```

```
mysql> SELECT * FROM sales_details;
+------------------+----------------------+---------------------+----------+
| sales_details_id | sales_transaction_id | products_product_id | quantity |
+------------------+----------------------+---------------------+----------+
|                1 |                    1 |                   1 |        1 |
|                2 |                    1 |                   2 |        1 |
|                3 |                    1 |                   3 |        1 |
+------------------+----------------------+---------------------+----------+
```

Citations

The database design and documentation were created by Arthur Adjamoglian and Min Deng. We received feedback from our TA, Razeel Hassan Ibrahim Rasheed, and eight peer reviewers: Pavel Gorshenkov, Ezri Martinez, Jacob Gervais, Thien Ngan Nguyen, Evan Neuenschwander, Christopher Long, Mina Go, and Paulo Mancilla. Their suggestions helped us improve our database structure, relationships, and documentation.